Autonomous Maze Navigation with an iRobot Create 3 Using LiDAR

Marjan Rahimof, Daniel Chiemezie Nwadike, Mark Rijpstra and Yehor Oblyvantsov

***Abstract*—A robot that drives through an unknown maze has to see the walls, choose where to go and get out of trouble on its own. This paper describes a navigation system for an iRobot Create 3 with a 2D LiDAR and a Raspberry Pi 4 running ROS 2 Jazzy. slam_toolbox maps the maze, an explorer node sends the robot to the edges of the known area, preferring the ones towards the finish, and Nav2 with an MPPI controller drives it there. In a simulation of the course maze the robot explored the whole maze in 2 minutes without touching a wall. On the real robot it reached the finish in 7 of 13 test runs, at most 0.11 m from it and within 178 s. Exploration on a SLAM map can run on the robot's own Raspberry Pi once the middleware, the message settings and the controller fit the Create 3.**

***Index Terms*—mobile robotics, maze navigation, SLAM, frontier exploration, ROS 2, LiDAR**

I. Introduction

A mobile robot has to sense what is around it, decide where to go and then act on that decision, over and over [1], [2]. A maze is a good test for this: walls, corridors and loops are easy to build and change, and it's easy to see whether the robot made it. One way to solve a maze is to build a map while driving. Simultaneous localisation and mapping (SLAM) estimates the robot's position and a map of the walls at the same time [3], and the robot explores by driving to frontiers, the borders between the mapped and the unknown part [4].

A. Problem statement

The task is to program an iRobot Create 3 to drive on its own from a start point to a finish point through a maze it hasn't seen before. The floor is flat, the walls are at about 90° to each other, and the maze can have loops. The start and the finish are only given a few minutes before the demonstration.

Our research question is: *can frontier exploration on a SLAM map, running only on the robot's Raspberry Pi 4, explore an unknown maze with loops and reach a finish given relative to the start within one 8-minute run?*

B. System description

Table I lists the hardware and software. The Create 3 publishes its sensor data and takes velocity commands over ROS 2 [5], [6]. The Raspberry Pi is connected to the Create 3 with USB-C and to the LiDAR with USB. It runs the LiDAR driver, SLAM, Nav2 and the explorer node.

All authors are with the Master Smart Systems Engineering, Hanze University of Applied Sciences, Groningen, The Netherlands. Introduction Project (SEVM25IP), Group 6.

Table I: Hardware and software used.

Component	Model / version	Role
Mobile base	iRobot Create 3	wheels, bumper, odometry
Computer	Raspberry Pi 4	runs SLAM, Nav2 and the explorer
LiDAR	Slamtec RPLiDAR A1M8	360° distance measurement [7]
Operating system	Ubuntu 24.04	on the Raspberry Pi
Middleware	ROS 2 Jazzy, Cyclone DDS	communication between nodes
Language	Python 3.12	explorer node, launch files

II. Design

The system that ran in the demonstration uses a map. While the robot drives, the LiDAR builds an occupancy grid, the explorer node picks the next goal on it, and the Nav2 navigation stack plans a path there and follows it [8]. It uses the LiDAR, the wheel odometry, the bumper and the battery state, and everything runs on the Raspberry Pi in the robot's ROS 2 namespace (Table II).

Table II: Software layers of the navigation system.

Layer	Software	Configuration
Sensing	rplidar_ros 2.1.0	360° scans, LiDAR mounted backwards
Mapping	slam_toolbox 2.8.4	online asynchronous mapping, 5 cm cells
Exploration	own explorer node	frontiers, goal choice, finish, wall alignment
Planning	Nav2 NavFn planner	path on the map, replanned once a second
Control	Nav2 MPPI controller	at most 0.15 m/s, no reversing
Safety	Nav2 collision monitor	stops 1.2 s before a predicted hit

A. Mapping

slam_toolbox matches each scan against the map to work out where the robot is, and keeps a grid of free, occupied and unknown cells [3]. The LiDAR is mounted backwards, so a static transform with a 180° yaw turns it around. At first no odometry or transforms arrived on the Pi. That only worked after switching from Fast DDS to Cyclone DDS, which is what the Create 3 runs [6]. The minimum travel thresholds are set to zero, so the map keeps getting new scans while the robot stands still (Table V).

B. Frontier exploration

A frontier is the border between known free space and unknown space [4]. On every new map a breadth-first search

starts at the robot and spreads through free cells. It gives every reachable cell c its path length $n(c)$. Reachable free cells next to unknown space make up the frontiers. Frontiers shorter than 0.4 m are dropped, because the robot is 0.35 m wide. On each frontier the goal candidate is the cell furthest from the walls, at least 0.25 m from them and 0.3 m from earlier goals. The next goal is

$$g^* = \arg\min_{g \in G} \left(n(g) + \|g - f\| \right), \tag{1}$$

where $\|g - f\|$ is the straight-line distance to the finish f. This is the same idea as the node cost in A* [9]: the robot explores in the direction of the finish. Without a finish only $n(g)$ counts. As soon as there's a reachable cell within 0.3 m of the finish, that cell becomes the goal. A goal is cancelled when its frontier gets explored on the way, and dropped after 90 s. The explorer stops after three bumps, at 20 % battery or when the time is up.

C. Alignment with the walls

Before exploring, the robot leaves the dock and turns until it's parallel to the walls, so the walls run along the map axes. Every window of seven scan points that is close to a straight line gives a direction φ_i and a length l_i. Multiplying the directions by four makes all right angles the same angle, so the wall direction is

$$\theta = \frac{1}{4} \operatorname{atan2}\left(\sum_i l_i \sin 4\varphi_i,\ \sum_i l_i \cos 4\varphi_i \right). \tag{2}$$

The robot only turns if at least 60 % of the wall length is within 10° of θ. It turns until $|\theta| \leq 1°$ and then resets the odometry and the map. A finish given as a distance and an angle from the start pose is then moved into the new map frame.

D. Navigation

Nav2 uses a configuration based on the TurtleBot 4, which has the same base and LiDAR [10]. The Create 3 sends its odometry as best effort, but Nav2 wants reliable messages [11], so a small relay node sends it on as reliable. The Create 3 only drives backwards a few centimetres [12], so reversing and the BackUp recovery are turned off. At first the controller was Regulated Pure Pursuit [13]. It stopped next to walls, because its collision check counted 3–7 cm of space as a hit. It was replaced by MPPI [14], which samples 1,000 trajectories 3 s ahead at up to 0.15 m/s and gives paths close to walls a high cost (Fig. 1). A collision monitor slows the robot down 1.2 s before a predicted hit.

E. Development

The system was built between 30 September and 20 October. Table III lists the problems found on the robot and how they were fixed.

Table III: Problems found during development and how they were fixed.

Problem on the robot	Cause	Fix
No odometry or transforms	Fast DDS on the Pi, Cyclone DDS on the robot	Cyclone DDS everywhere
Nav2 gets no odometry	best effort vs. reliable QoS	relay node
Explorer uses a third of a core, Pi throttles at 83 °C	Nav2 feedback at 100 Hz, two explorers running	feedback at 20 Hz, lighter node
Robot stops at walls and in doorways	pure pursuit collision check	MPPI

III. Methods

A. Tests without the robot

The frontier search and the wall direction code are plain Python. They are tested with 32 pytest cases on small hand-drawn maps and made-up scans. The ROS nodes were tested without taking up the shared robot. A 271 s recording of a drive through the maze, made on the robot on 2 October, was played back through slam_toolbox in a separate ROS domain. The explorer's handling of Nav2 goals was checked against a fake Nav2 action server.

B. Simulation

Nav2 was tuned in a small 2D simulator written for this project. It stands in for the Create 3 and the LiDAR inside the map of the course maze from 12 October. It publishes odometry, transforms, scans and the dock, battery and bump topics, answers the undock, rotate and reset requests, and treats the robot as a 0.171 m disc that stops and reports a bump when it drives into a wall. slam_toolbox, Nav2 and the explorer ran on top of it without changes, in a separate ROS domain. The original pure pursuit settings, a tuned pure pursuit, pure pursuit without its collision check and several MPPI settings were compared.

C. Runs on the robot

On 20 October there were 13 runs with a finish in the course maze, nine starting on the dock and four on the floor. The finish was given as a distance and an angle from the start pose. Every run was recorded as a rosbag with the map, pose, odometry, scans, transforms, Nav2 plan, velocity commands, goals and bump events. A monitor script logged the Raspberry Pi's temperature, the battery and the distance to the finish every 10 s, and would stop a run above 82 °C. These runs used the development version of the explorer from 20 October.

D. Measurements

To compare SLAM settings, one recorded scan with a fixed pose was played to slam_toolbox in four configurations, and the cells that stayed unknown at 0–1, 1–2, 2–3 and 3–4 m from the robot were counted. The CPU load of the explorer was measured on the Raspberry Pi; for the Nav2 feedback rate this was done against a fake Nav2 server. The code that ran in the demonstration has the tag demo-2026-10-23 at https://github.com/yeoblyv/corrida.



Fig. 1: Simulated runs in the map of the course maze. Left: pure pursuit, stuck after 100 s. Middle: MPPI, whole maze in 2 min. Right: MPPI with a finish, reached in 64 s. Blue: start, green: end, red: path.

IV. Results

In the simulation (Fig. 1) the original pure pursuit settings got 3.2 m in 4 min and got stuck after 100 s. With MPPI the robot explored the whole maze in 2 min, drove 10.5 m and never touched a wall. With a finish, it got there in 64 s.

On the robot the finish was reached in 7 of the 13 runs (Table IV), 0.02–0.11 m from it and 40–178 s after the explorer started, with a median of 57 s. One run stopped 0.38 m from the finish, two stopped after bumps, and in three the explorer found no frontier left before it found the finish. Fig. 2 shows the longest successful run. The highest Pi temperature per run was 72–81 °C. Table V has the other measurements.

Table IV: Runs on the robot on 20 October. Time is counted from the explorer's first action. Distance is to the finish at the end of the run; for runs that didn't reach it, it's the closest distance in the 10 s monitor log.

Run	Start	Result	Time (s)	Distance (m)
dock1	dock	no frontier left	33	3.94
dock2	dock	no frontier left	53	2.08
dock3	dock	finish reached	178	0.02
dock4	dock	finish reached	64	0.04
dock5	dock	finish reached	102	0.11
dock9	dock	stopped after bumps	115	1.38
dock10	dock	stopped near finish	130	0.38
dock11	dock	finish reached	51	0.05
mark6	floor	stopped after bumps	90	1.29
mark7	floor	finish reached	45	0.08
mark8	floor	finish reached	40	0.04
spot1	dock	no frontier left	102	1.36
spot2	floor	finish reached	57	0.05

Setting min_pass_through from 2 to 1 on top of zero travel thresholds didn't change the share of unknown cells.

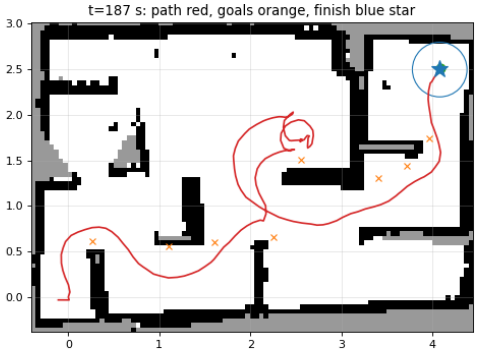


Fig. 2: Map at the end of run dock3: path (red), goals (orange) and finish (star) with the 0.3 m arrival circle. The robot stopped 0.02 m from the finish.

V. Discussion and Conclusion

The results show that the system only works on the Raspberry Pi once three things fit the Create 3: the middleware, the reliability of the odometry topic and the controller. Most of the CPU load came from Nav2's goal feedback and from handling every scan and odometry message. After fixing that, one explorer used about a tenth of a core, and the Pi stayed below 82 °C in all runs. Pure pursuit counted a few centimetres of space as a hit and got stuck. MPPI looks at the distance

Table V: Results of the tests and measurements.

Measurement	Result
Unit tests, frontier search and wall direction	32 of 32 passing
Unknown cells 3–4 m away, standing robot, travel thresholds 0.2 m / 0	95.7 % / 15.9 %
Explorer CPU load, Nav2 feedback at 100 Hz / 20 Hz	about 1/3 / 1/10 of a core
Explorer CPU load, scan and odometry always / only during set-up	32 % / 18 % of a core
Pi temperature with two explorers running	83 °C, throttling
Space at which pure pursuit stopped on the robot	3–7 cm

to the walls along its sampled trajectories, and it explored the whole simulated maze.

On the robot, frontier search with the finish term in 1 reached the finish in about half of the runs, always within 3 minutes and closer than 0.11 m. Three runs ended with no frontier left before the finish was found, two stopped after bumps and one stopped 0.38 m short of the finish.

There are two more limits. slam_toolbox gets slower in long runs, so the bringup is restarted before every run. The finish is placed using odometry, so an error of 1° in the start heading moves a finish 5 m away by about 9 cm. Overall, frontier exploration on a SLAM map with Nav2 works on the Create 3's own Raspberry Pi 4.

ACKNOWLEDGEMENTS

Table VI: Contribution of each group member. Share: non-blank lines each member added to the project repository, all branches.

Member	Contribution	Share (%)
Marjan Rahimof Daniel Chiemezie Nwadike Mark Rijpstra		
Yehor Oblyvantsov	Repository and workflow; LiDAR, SLAM and Nav2 setup; frontier explorer; simulator and robot tests; the navigation system used in the demonstration; Sections II–V of this report	62

REFERENCES

[1] M. Ben-Ari and F. Mondada, *Elements of robotics*. Cham: Springer, 2018.

[2] P. Corke, *Robotics, vision and control: Fundamental algorithms in python*, 3rd ed. Cham: Springer, 2023.

[3] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.

[4] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE int. Symp. Computational intelligence in robotics and automation (CIRA'97)*, 1997, pp. 146–151.

[5] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[6] iRobot Education, "iRobot Create 3 documentation." https://iroboteducation.github.io/create3_docs/, 2026.

[7] Slamtec, "RPLIDAR A1 low cost 360 degree laser range scanner." https://www.slamtec.com/en/Lidar/A1, 2026.

[8] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ int. Conf. Intelligent robots and systems (IROS)*, 2020, pp. 2718–2725.

[9] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[10] Clearpath Robotics, "turtlebot4, version 2.1.1." https://github.com/turtlebot/turtlebot4, 2025.

[11] Open Robotics, "Quality of service settings, ROS 2 documentation: Jazzy." https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html, 2026.

[12] iRobot Education, "Create 3 safety overrides." https://iroboteducation.github.io/create3_docs/api/safety/, 2026.

[13] S. Macenski, S. Singh, F. Martín, and J. Ginés, "Regulated pure pursuit for robot path tracking," *Autonomous Robots*, vol. 47, no. 6, pp. 685–694, 2023.

[14] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information-theoretic model predictive control: Theory and applications to autonomous driving," *IEEE Transactions on Robotics*, vol. 34, no. 6, pp. 1603–1622, 2018.